

1. Traditional File-Based System

Definition

A Traditional File-Based System is a system where data is stored in separate files and each application program manages its own data.

In this system, files are stored on the computer, and programs access these files directly.

Each department or application keeps its own files, and there is no central control of data.

Example

Suppose a college has different departments.

- Admission Department → Student File
- Accounts Department → Fee File
- Library → Library File

Example files:

Student File

Roll_No	Name	Department
101	Rahul	CSE
102	Anjali	ECE

Fee File

Roll_No	Fee_Status
101	Paid
102	Pending

Library File

Roll_No	Book_Issued
101	DBMS
102	None

Each department keeps separate files.

Problems in Traditional File System

1. Data Redundancy

Same data is stored in multiple files.

Example

Student name may appear in many files.

2. Data Inconsistency

If data is updated in one file but not in another.

Example

Student name changed in admission file but not in library file.

3. Data Isolation

Data is stored in many different files, making it difficult to access together.

4. Security Problems

No proper control over who can access data.

5. Difficult Data Access

To get information, a new program must be written.

6. Poor Data Sharing

Different departments cannot easily share data.

2. Database Management System (DBMS)

Definition

A Database Management System (DBMS) is a software system used to store, manage, and organize data in a centralized database.

It allows multiple users and applications to access and manage data efficiently.

Examples of DBMS:

- MySQL
 - Oracle
 - PostgreSQL
 - SQL Server
 - MongoDB
-

Example

Instead of separate files, the college stores data in one database.

Student Table

Roll_No	Name	Department	Fee_Status	Book_Issued
101	Rahul	CSE	Paid	DBMS
102	Anjali	ECE	Pending	None

Now all departments access the same database.

Features of DBMS

1. Centralized Data

All data is stored in one database.

2. Reduced Redundancy

Duplicate data is minimized.

3. Data Consistency

If data is updated, it is updated everywhere.

4. Security

DBMS provides user authentication and access control.

5. Data Sharing

Multiple users can access the database simultaneously.

6. Backup and Recovery

Data can be recovered if a system failure occurs.

3. Difference Between File System and DBMS

Feature	File-Based System	DBMS
Data Storage	Separate files	Centralized database
Redundancy	High	Low
Data Consistency	Difficult to maintain	Maintained easily

Feature	File-Based System	DBMS
Data Sharing	Limited	Easy
Security	Poor	Strong security
Data Access	Requires programming	Use SQL queries
Backup	Difficult	Easy

4. Simple Real-Life Analogy (Good for Demo Class)

Imagine a hospital.

File-Based System

Each department keeps separate patient records in files.

- Reception file
- Doctor file
- Billing file

Information may be repeated and inconsistent.

DBMS

All patient data is stored in one centralized database.

Doctors, nurses, and billing staff can access the same information.

5. Short Conclusion (Good for Ending Demo)

A Traditional File-Based System stores data in separate files managed by different programs, which causes redundancy and inconsistency.

A Database Management System (DBMS) stores data in a centralized database and provides efficient data management, security, and easy access.

2. Functional Dependency (FD) in DBMS

1. Introduction

In a **database**, data is stored in tables (relations).

Sometimes **one column determines the value of another column**.

This relationship between attributes is called **Functional Dependency**.

Definition:

A **Functional Dependency (FD)** is a relationship between two attributes in a table where **one attribute uniquely determines another attribute**.

It is written as:

$X \rightarrow Y$

Where:

- **X** = determinant (left side)
- **Y** = dependent attribute (right side)

Meaning:

If we know **X**, we can determine **Y**.

2. Simple Example

Consider a **Student Table**

Student_ID	Name	Department	Phone
101	Rahul	CSE	9876543210
102	Anjali	ECE	9876500000
103	Amit	CSE	9876511111

Here:

Student_ID $\rightarrow$ Name

Student_ID $\rightarrow$ Department

Student_ID $\rightarrow$ Phone

Meaning:

If we know **Student_ID**, we can determine:

- Name
- Department
- Phone

So **Student_ID** functionally determines other attributes.

3. Key Idea of Functional Dependency

Functional dependency shows:

- Relationship between attributes
- How data depends on other data
- Helps remove redundancy
- Helps perform **normalization**

Example:

If **Employee_ID** $\rightarrow$ **Employee_Name**

Then for every Employee_ID there will be **only one Employee_Name**.

4. Types of Functional Dependency

1. Trivial Functional Dependency

A dependency where the **right side attribute is part of the left side attribute**.

Form:

X $\rightarrow$ **Y** where **Y** $\subseteq$ **X**

Example:

Student_ID **Name**

FD:

{Student_ID, Name} $\rightarrow$ **Name**

Because **Name** is already part of the left side.

So it is **trivial**.

2. Non-Trivial Functional Dependency

When **right side attribute is NOT part of left side attribute**.

Example:

Student_ID $\rightarrow$ **Name**

Here:

Name is not part of Student_ID.

So it is **Non-Trivial FD**.

3. Completely Non-Trivial Functional Dependency

When **X and Y have no common attributes**.

Example:

Employee_ID → Salary

No attribute is common.

So it is **Completely Non-Trivial FD**.

5. Partial Dependency

Occurs when:

A **non-prime attribute depends on only part of a composite key**.

Example Table:

Student_ID Course_ID Grade

101	C1	A
-----	----	---

101	C2	B
-----	----	---

Composite Key = **(Student_ID, Course_ID)**

If:

Student_ID → Student_Name

Then the dependency is **partial**, because it depends only on **Student_ID**, not the whole key.

This causes **data redundancy**.

6. Transitive Dependency

Occurs when:

A → B

B → C

Then:

A → C

Example:

Student_ID Department_ID Department_Name

Dependencies:

Student_ID → Department_ID

Department_ID → Department_Name

Therefore:

Student_ID → Department_Name

This is called **Transitive Dependency**.

7. Why Functional Dependency is Important

Functional dependency is important because it helps in:

1. Database Design

Helps design efficient database tables.

2. Normalization

Used to remove:

- redundancy
- duplication

3. Data Consistency

Ensures correct data relationships.

4. Identify Keys

Helps determine:

- Primary key
- Candidate key

8. Real-Life Example

Consider a **University Database**

Roll_No Student_Name Course Teacher

Possible FD:

Roll_No $\rightarrow$ Student_Name

Course $\rightarrow$ Teacher

Meaning:

- Roll number determines student name.
- Course determines teacher.

9. Mathematical Representation

Functional Dependency:

$X \rightarrow Y$

Meaning:

For any two tuples **t1 and t2** in relation **R**:

If:

$$t1[X] = t2[X]$$

Then:

$$t1[Y] = t2[Y]$$

Meaning:

If X values are same, Y values must also be same.

10. Summary (Good Ending for Demo)

Functional Dependency is a rule in DBMS that defines **how attributes depend on each other**.

It helps to:

- identify relationships between attributes
- reduce data redundancy
- design better databases
- perform normalization

Basic notation:

$$X \rightarrow Y$$

Meaning:

X determines Y

Short Demo Line (Good for presentation ending):

Functional Dependency helps us understand how data is related in a table and plays a major role in normalization and efficient database design.

3. Normalization in DBMS

1. Definition

Normalization is the process of **organizing data in a database to reduce redundancy and improve data integrity**.

In simple words:

Normalization helps us **remove duplicate data and organize tables properly**.

It divides large tables into **smaller related tables** and creates relationships between them.

2. Why Normalization is Needed

Without normalization, databases can have many problems.

1. Data Redundancy

Same data is stored many times.

Example

Student name repeated many times.

2. Data Inconsistency

If one value changes but others are not updated.

Example

Teacher name updated in one row but not in others.

3. Insert Anomaly

We cannot insert some data without other data.

Example

Cannot add a new course unless a student enrolls.

4. Update Anomaly

Updating the same data in many rows.

5. Delete Anomaly

Deleting one record may remove important information.

3. Example Before Normalization

Student Course Table

Student_ID	Student_Name	Course	Teacher
101	Rahul	DBMS	Sharma
101	Rahul	OS	Verma
102	Anjali	DBMS	Sharma

Problems:

- Student name repeated
 - Teacher repeated
 - Data redundancy
-

4. After Normalization

We divide the table into smaller tables.

Student Table

Student_ID Student_Name

101	Rahul
102	Anjali

Course Table

Course_ID Course_Name Teacher

C1	DBMS	Sharma
C2	OS	Verma

Enrollment Table

Student_ID Course_ID

101	C1
101	C2
102	C1

Now:

- No repeated data
 - Tables are organized
-

5. Normal Forms

Normalization is done in stages called **Normal Forms (NF)**.

Main normal forms:

- 1 First Normal Form (1NF)
 - 2 Second Normal Form (2NF)
 - 3 Third Normal Form (3NF)
-

6. First Normal Form (1NF)

A table is in **1NF** if:

- Each column contains **atomic (single) values**
- No repeating groups
- Each record is unique

Example (Not in 1NF)

Student_ID Name Subjects

101	Rahul	DBMS, OS
-----	-------	----------

Here subjects contain multiple values.

Convert to 1NF:

Student_ID Name Subject

101	Rahul	DBMS
-----	-------	------

101	Rahul	OS
-----	-------	----

7. Second Normal Form (2NF)

A table is in **2NF** if:

- It is already in **1NF**
- No **partial dependency**

Meaning:

Non-key attributes must depend on the **entire primary key**.

Example

Student_ID Course_ID Student_Name

Primary key = (Student_ID, Course_ID)

But:

Student_ID → Student_Name

So this is **partial dependency**.

Solution:

Create two tables.

Student Table

| Student_ID | Student_Name |

Course Enrollment Table

| Student_ID | Course_ID |

8. Third Normal Form (3NF)

A table is in **3NF** if:

- It is already in **2NF**
- No **transitive dependency**

Example

| Student_ID | Department_ID | Department_Name |

Dependencies:

Student_ID → Department_ID

Department_ID → Department_Name

So:

Student_ID → Department_Name (transitive)

Solution:

Student Table

| Student_ID | Department_ID |

Department Table

| Department_ID | Department_Name |

9. Advantages of Normalization

1. Reduces Data Redundancy

Less duplicate data.

2. Improves Data Consistency

Updates occur in one place.

3. Saves Storage Space

Less repeated data.

4. Better Database Design

Tables become structured and easier to manage.

5. Prevents Data Anomalies

Avoids insert, update, and delete problems.

10. Short Summary (Good for Ending Demo)

Normalization is a process used in DBMS to **organize data into multiple tables to reduce redundancy and improve data consistency**.

The main normal forms are:

- **1NF – Remove repeating groups**
- **2NF – Remove partial dependency**
- **3NF – Remove transitive dependency**

Normalization helps create **efficient and reliable databases**.

4. Transaction in DBMS

1. Definition

A **Transaction** in DBMS is a **sequence of operations performed on a database as a single logical unit of work**.

A transaction must either:

- **complete fully, or**
- **not execute at all**

This ensures the database remains **correct and consistent**.

In simple words:

A transaction is a group of database operations that must be executed **completely or not at all**.

2. Example of Transaction

Consider a **bank money transfer**.

Rahul transfers **₹1000** to Aman.

The transaction involves two operations:

1. Deduct ₹1000 from Rahul's account
2. Add ₹1000 to Aman's account

Transaction steps:

Step 1: Rahul balance = Rahul balance – 1000

Step 2: Aman balance = Aman balance + 1000

If **both steps succeed**, the transaction is **completed**.

If **one step fails**, the transaction must **cancel completely**.

Otherwise the database becomes incorrect.

Example problem:

- Money deducted from Rahul
- But not added to Aman

This creates **data inconsistency**.

DBMS prevents this using **transactions**.

3. Transaction States

A transaction goes through several states.

1. Active

Transaction is currently executing.

2. Partially Committed

All operations executed but not yet permanently saved.

3. Committed

Transaction successfully completed and changes saved permanently.

4. Failed

Transaction cannot proceed due to error.

5. Aborted

Transaction is cancelled and database returns to previous state.

4. ACID Properties of Transaction

Transactions follow **ACID properties** to maintain database reliability.

1. Atomicity

Atomicity means **all operations occur or none occur**.

Example:

Money transfer must complete both steps.

If one fails → transaction rolls back.

2. Consistency

After a transaction, the database must remain **valid and correct**.

Example:

Total balance before and after transfer must remain correct.

3. Isolation

Multiple transactions should **not interfere with each other**.

Example:

Two users transferring money at the same time should not affect each other.

4. Durability

Once a transaction is **committed**, changes are **permanently stored**.

Even if the system crashes, the data remains saved.

5. Transaction Control Commands (TCL)

DBMS provides commands to manage transactions.

1. COMMIT

Saves all changes permanently.

Example:

COMMIT;

2. ROLLBACK

Cancels the transaction and returns database to previous state.

ROLLBACK;

3. SAVEPOINT

Creates a checkpoint inside a transaction.

SAVEPOINT sp1;

6. Real Life Example

Consider **online shopping**.

Transaction steps:

1. User places order
2. Payment deducted
3. Order confirmed

If payment fails → order is cancelled.

DBMS ensures the process happens correctly using transactions.

7. Simple Diagram for Explanation

Transaction flow:

Start Transaction



Operations



Success ?



Yes No



Commit Rollback

8. Short Summary (Good Ending for Demo)

A **Transaction in DBMS** is a set of database operations that are executed as a **single unit of work**.

Transactions ensure data reliability using **ACID properties**:

- Atomicity
- Consistency
- Isolation
- Durability

They help maintain **data integrity and correctness** in a database system.

5. Joins in SQL

1. Definition

A **JOIN** in SQL is used to **combine rows from two or more tables based on a related column between them**.

In simple words:

A **JOIN** allows us to retrieve data from multiple tables at the same time.

Usually, tables are connected using a **common column (key)**.

2. Example Tables

Student Table

Student_ID	Name
101	Rahul
102	Anjali
103	Amit

Course Table

Student_ID	Course
101	DBMS
102	OS
104	Networks

Here **Student_ID** is the common column.

3. Types of SQL Joins

Main types of joins:

1 INNER JOIN

2 LEFT JOIN

3 RIGHT JOIN

4 FULL JOIN

4. INNER JOIN

INNER JOIN returns only the matching records from both tables.

Example:

```
SELECT Student.Name, Course.Course
FROM Student
```

INNER JOIN Course
ON Student.Student_ID = Course.Student_ID;

Result:

Name Course

Rahul DBMS

Anjali OS

Student **Amit (103)** is not shown because there is no matching course.

5. LEFT JOIN

LEFT JOIN returns all records from the left table and matching records from the right table.

If no match exists, it shows **NULL**.

```
SELECT Student.Name, Course.Course
FROM Student
LEFT JOIN Course
ON Student.Student_ID = Course.Student_ID;
```

Result:

Name Course

Rahul DBMS

Anjali OS

Amit NULL

6. RIGHT JOIN

RIGHT JOIN returns all records from the right table and matching records from the left table.

```
SELECT Student.Name, Course.Course
FROM Student
RIGHT JOIN Course
ON Student.Student_ID = Course.Student_ID;
```

Result:

Name Course

Rahul DBMS

Anjali OS

Name Course

NULL Networks

Student_ID **104** exists only in the Course table.

7. FULL JOIN

FULL JOIN returns all records when there is a match in either table.

```
SELECT Student.Name, Course.Course
FROM Student
FULL JOIN Course
ON Student.Student_ID = Course.Student_ID;
```

Result:

Name Course

Rahul DBMS

Anjali OS

Amit NULL

NULL Networks

8. Why Joins are Important

Joins are important because:

- Data in databases is usually stored in **multiple tables**
 - Joins allow us to **retrieve related data**
 - They help maintain **database normalization**
 - They reduce **data redundancy**
-

9. Real-Life Example

Consider a **company database**.

Employee Table

Emp_ID Name

1 Ravi

2 Neha

Department Table

Emp_ID Department

1 HR

2 IT

Using **JOIN**, we can get:

Name Department

Ravi HR

Neha IT

10. Short Summary (Good Ending for Demo)

A **JOIN in SQL** is used to combine data from multiple tables based on a related column.

Common types of joins:

- **INNER JOIN** – returns matching records
- **LEFT JOIN** – returns all records from left table
- **RIGHT JOIN** – returns all records from right table
- **FULL JOIN** – returns all records from both tables

Joins help retrieve related data efficiently from relational databases.